

TEC0738 - Desenho de Aplicativos para Dispositivos Móveis

Semana 4 – Ciclo de Vida e Estado Simples

Da interface estática à reatividade dinâmica.

Objetivos da Aula

1 

Compreender o Ciclo de Vida dos componentes.

2 

Diferenciar StatelessWidget de StatefulWidget.

3 

Implementar Estado Simples com a função `setState()`.

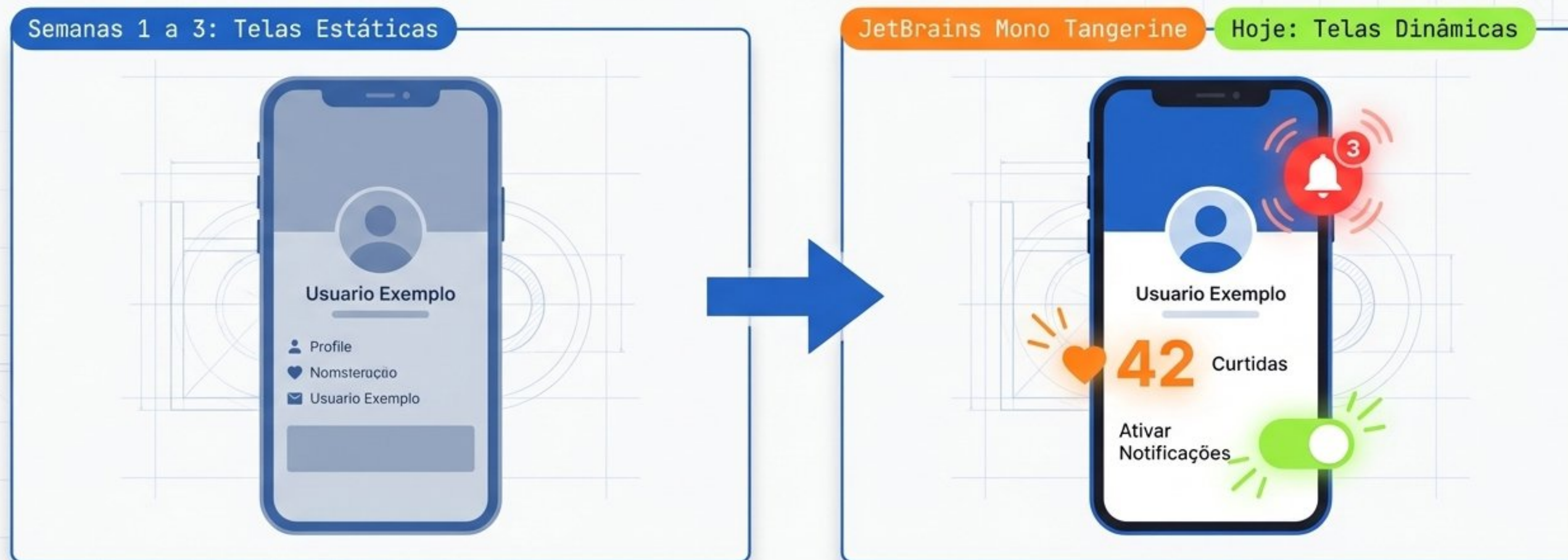
4 

Dominar os três métodos estruturais: `initState()`, `build()` e `dispose()`.

O Ponto de Virada

Nas semanas anteriores, construímos a “lataria”: interfaces visuais de apresentação e organização geométrica.

Hoje, vamos instalar o “motor”. A interface passará a reagir às interações do usuário em tempo real.



O que é 'Estado'?

Estado são os dados que mudam ao longo do tempo na interface. Se um componente precisa ser redesenhado após a tela carregar, ele possui Estado.

1. Contador numérico



0 → 1

2. Entrada de texto



3. Alternância (Toggle)

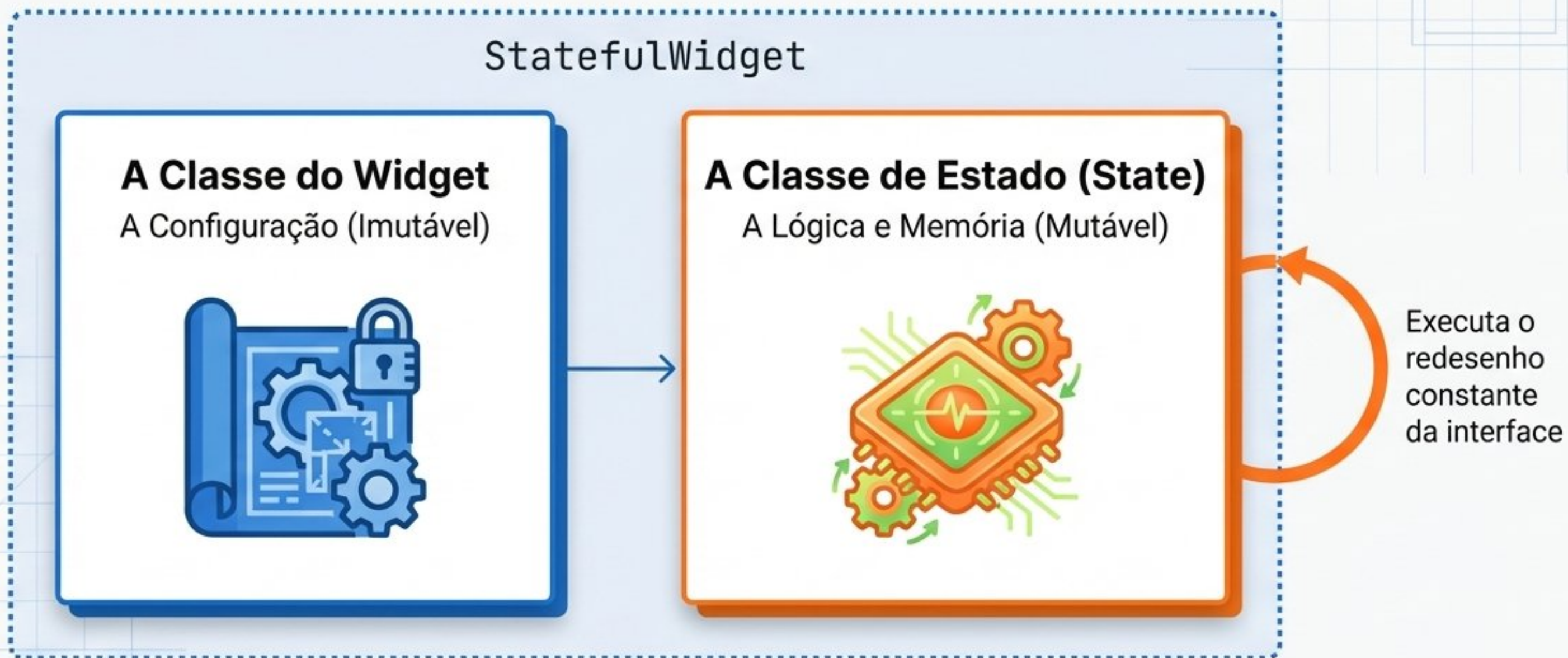


O Paradigma dos Widgets

StatelessWidget 	StatefulWidget 
Natureza Sem Estado / Imutável	Natureza Com Estado / Dinâmico e Mutável
Arquitetura Apenas 1 Classe principal	Arquitetura Dividido em 2 Classes
A Metáfora Uma Fotografia (Uma vez impressa, não muda)	A Metáfora Um Vídeo (Uma sucessão de quadros atualizados)
Casos de Uso Textos fixos, ícones decorativos, layouts estruturais.	Casos de Uso Formulários, contadores, animações, botões.

A Anatomia do StatefulWidget

Diferente dos componentes estáticos, um widget interativo exige uma separação de responsabilidades pelo framework Flutter.



O Eletrocardiograma do Widget

O framework gerencia rigidamente o que acontece com a sua tela.

1. Nascimento

Inserção do widget na árvore do aplicativo.

`initState()`

2. Vida Ativa

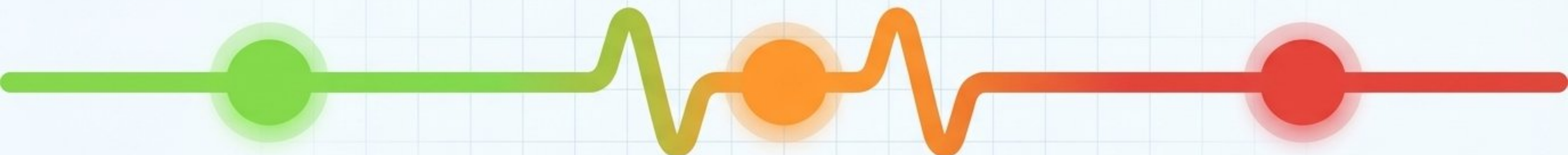
Interação contínua do usuário, renderizações e atualizações visuais.

`build()`
`setState()`

3. Morte

Remoção permanente da tela da memória do dispositivo.

`dispose()`



O Início de Tudo: initState()



- **Quando é acionado?** Exatamente uma única vez quando o widget é criado e inserido na tela.
- **Para que serve?**
 - Inicializar variáveis (ex: zerar um contador).
 - Conectar-se a bancos de dados ou streams de dados.
 - Preparar configurações que a tela precisará antes de aparecer.

Regra de Ouro: Nunca tente desenhar elementos visuais (UI) dentro deste método.

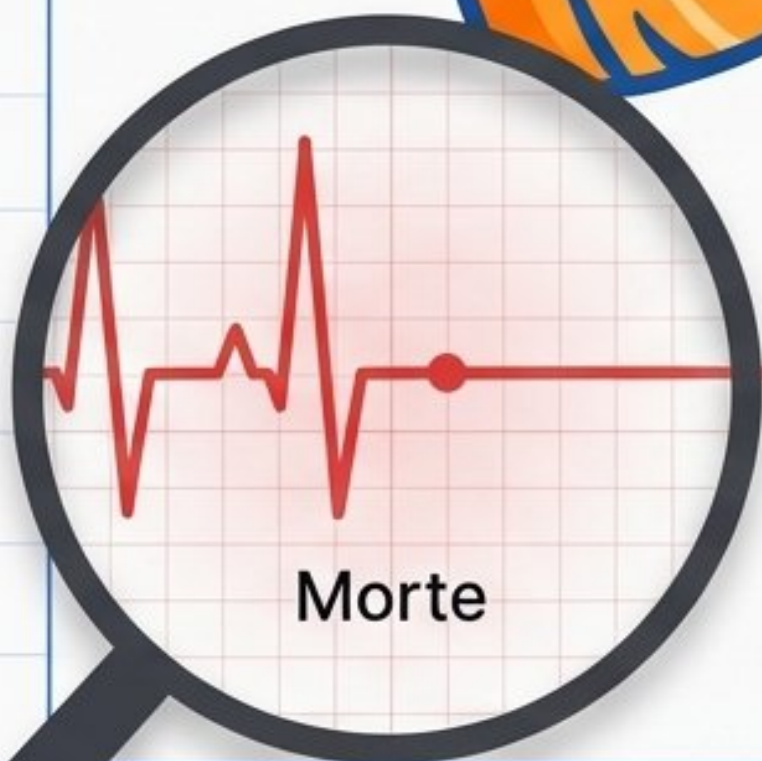
A Fábrica Visual: build() in Inter



- **Quando é acionado?** Toda vez que o widget precisa ser renderizado pela primeira vez ou atualizado visualmente.
- **Para que serve?** É aqui que a árvore de widgets (textos, botões, cores) é efetivamente desenhada na tela do celular.

Regra de Ouro: É o método mais executado do Flutter. Ele deve ser 'puro': focado apenas na interface. Nunca coloque chamadas pesadas de internet ou banco de dados aqui, ou seu aplicativo irá travar.

O Adeus e a Limpeza: dispose()

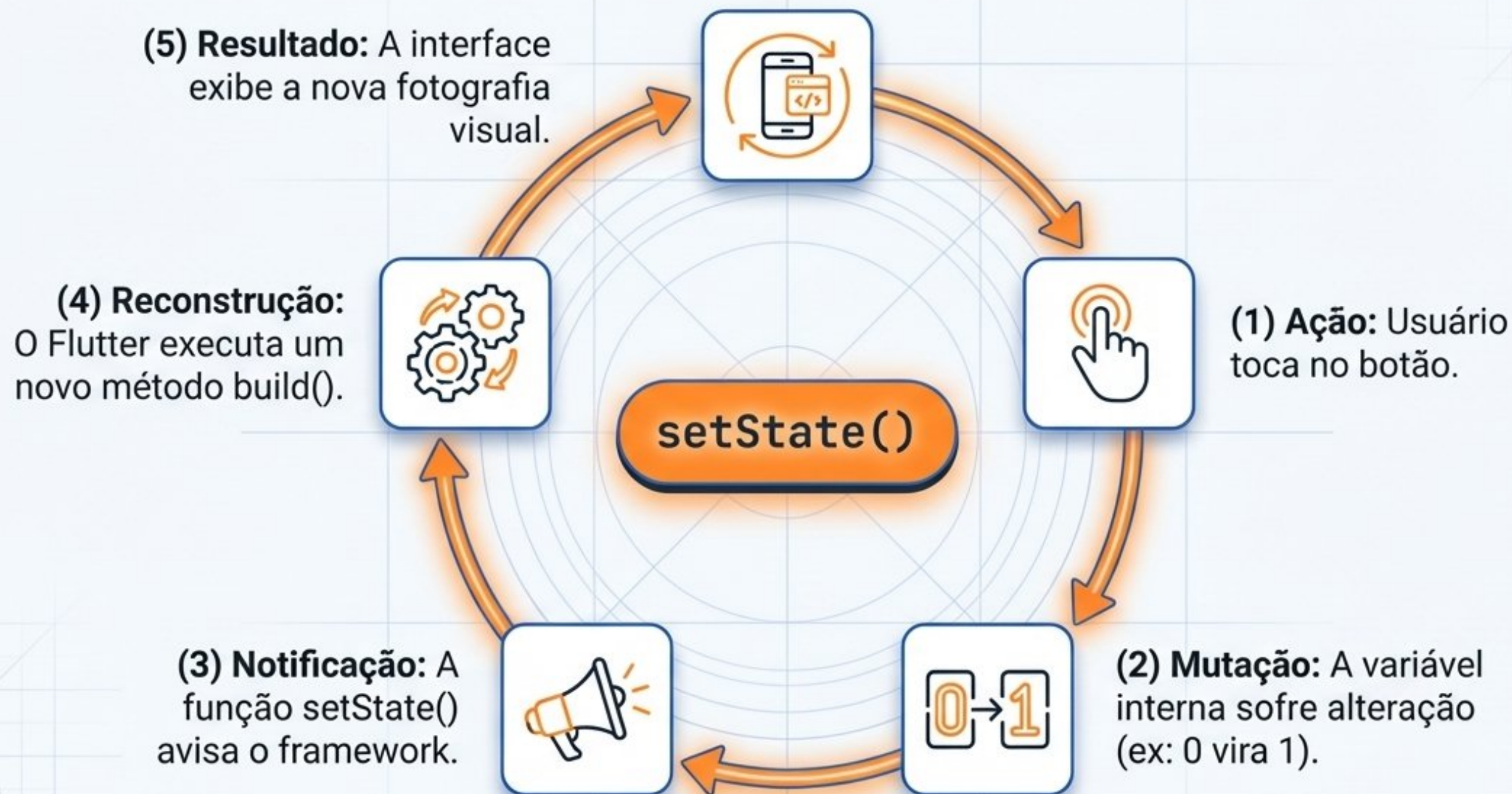


Memory Leak
`dispose()`

- **Quando é acionado?** Quando o widget é removido permanentemente (ex: quando o usuário aperta 'Voltar' e fecha a tela atual).
- **Para que serve?** Cancelar processos em segundo plano, parar animações e limpar a memória do dispositivo.
- **Por que importa?** Ignorar o `dispose()` deixa processos órfãos rodando invisivelmente. Com o tempo, isso causa o temido *Memory Leak* (vazamento de memória), drenando a bateria e travando o aplicativo.

O Motor da Interatividade: O Loop do `setState()`

Como avisar ao Flutter que um dado mudou e a tela precisa ser redesenhada?



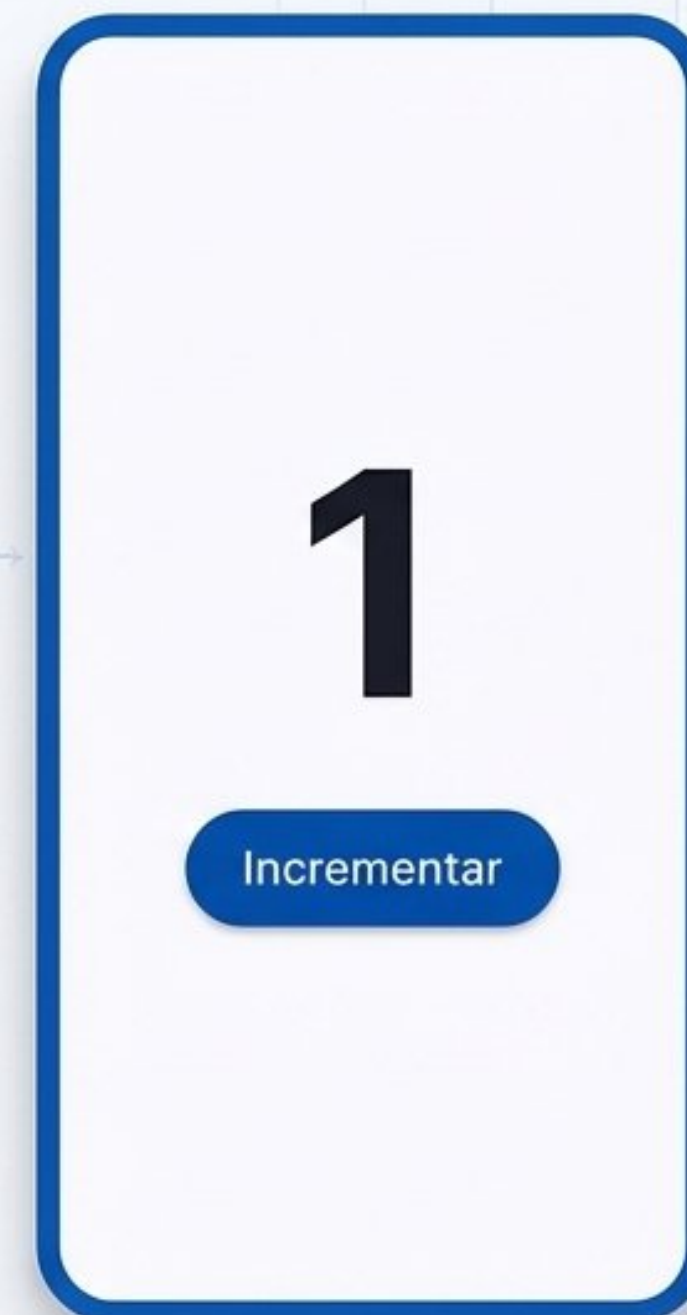
Prática Guiada: O App Contador

```
CounterApp.dart ×

Variável Inicial: int _contador = 0;

// Ação do Botão:
onPressed: () {
  setState(() {
    _contador++;
  });
}

// O método build() lê o novo valor e atualiza a tela automaticamente:
Text(_contador.toString())
```



Raio-X do Sistema (Técnica de Debug)

Como visualizar o Ciclo de Vida agindo de forma invisível?

Adicione a função **print()** dentro do **initState()**, **build()** e **dispose()**.

```
@override
void initState() {
  super.initState();
  print('Executando initState');
}
```

```
@override
void dispose() {
  print('Executando dispose');
  super.dispose();
}
```

```
> flutter run
...
Executando initState
Executando build
Executando build
```

O Insight: Ao clicar no botão de incrementar, observe o terminal. Apenas o **build** aparece repetidas vezes. O **initState** ficou no passado, executado apenas na primeira renderização.

Atividade da Semana: Contador Avançado

Missão: Estender a inteligência do App Contador desenvolvido em aula.

Requisitos de Implementação

- ☐ Adicionar um novo botão dedicado ao decremento numérico.
- ☐ Criar uma trava de segurança: o contador nunca pode ser menor que 0.
- ☐ Adicionar lógica condicional dentro do `build()` para exibir um texto especial de cor diferente ('Meta atingida!') exatamente quando o contador chegar ao valor 10.

Meta atingida!

10

-

+



Explique, em poucas linhas, em quais situações práticas você deve optar por construir um StatefulWidget em vez de um StatelessWidget.